SkillNavora

PDF 14

Quick Reference and Cheat Sheets

Excel, SQL, DAX, Python, pandas, statistics, ML metrics, APIs and MCP, AI and LLM terms, Git and the command line.

Part of the AI for Data Engineers (Data Science & AI) programme.

Varun Kumar · SkillNavora · skillnavora.in
Edition: November 2026. Official links point to the provider's own site. The sandbox used to write this could not open them, so check each link before you share it.

Quick Reference and Cheat Sheets

The formulas, commands and terms used across the course, on one page each. Exact syntax can differ between tool versions; check the official documentation for your version.

Excel formulas

The formulas analysts use most. Ranges like A2:A100 are examples; adjust them to your sheet.

Item	What it does	Example
SUMIFS	Adds values that meet one or more conditions	`=SUMIFS(D:D, B:B, "North", C:C, ">=2025-01-01")`
COUNTIFS	Counts rows that meet conditions	`=COUNTIFS(B:B, "North", E:E, "Closed")`
AVERAGEIFS	Averages values that meet conditions	`=AVERAGEIFS(D:D, B:B, "South")`
XLOOKUP	Finds a value and returns a match from another column; exact match by default	`=XLOOKUP(G2, A:A, C:C, "Not found")`
INDEX + MATCH	Lookup that works in every Excel version	`=INDEX(C:C, MATCH(G2, A:A, 0))`
IF / IFS	Returns one value or another depending on a test	`=IFS(D2>=90, "A", D2>=75, "B", TRUE, "C")`
IFERROR	Replaces an error with a value you choose	`=IFERROR(D2/E2, 0)`
TEXTJOIN	Joins text with a separator, skipping blanks	`=TEXTJOIN(", ", TRUE, A2:C2)`
TEXTSPLIT	Splits text into cells (Microsoft 365)	`=TEXTSPLIT(A2, ",")`
LEFT / RIGHT / MID	Takes characters from text	`=MID(A2, 4, 3)`
TRIM / CLEAN	Removes extra spaces and non-printing characters	`=TRIM(CLEAN(A2))`
DATEDIF / EOMONTH	Date difference; last day of a month	`=EOMONTH(A2, 0)`
UNIQUE / SORT / FILTER	Dynamic arrays (Microsoft 365) that spill results	`=SORT(FILTER(A2:D100, D2:D100>1000), 4, -1)`
LET	Names a value inside a formula so it is calculated once	`=LET(t, SUM(D:D), D2/t)`
PivotTable	Summarises rows by groups without formulas	`Insert > PivotTable; Rows = Region, Values = Sum of Sales`
Power Query	Repeatable import and cleaning steps	`Data > Get Data; record steps, then Refresh`
Absolute reference	$ keeps a row or column fixed when copying	`=D2*$H$1`
Data validation	Restricts what can be typed in a cell	`Data > Data Validation > List`

SQL commands

Standard SQL that works in most databases. Exact function names can differ between PostgreSQL, MySQL, SQL Server and others.

Item	What it does	Example

SELECT ... FROM ... WHERE	Reads rows and filters them	`SELECT name, city FROM customers WHERE city = 'Pune';`
ORDER BY / LIMIT	Sorts and caps the rows returned	`SELECT * FROM orders ORDER BY amount DESC LIMIT 10;`
GROUP BY / HAVING	Aggregates per group; HAVING filters groups	`SELECT city, COUNT(*) FROM customers GROUP BY city HAVING COUNT(*) > 50;`
Aggregates	COUNT, SUM, AVG, MIN, MAX	`SELECT AVG(amount) FROM orders;`
INNER JOIN	Rows that match in both tables	`SELECT o.id, c.name FROM orders o JOIN customers c ON c.id = o.customer_id;`
LEFT JOIN	All rows from the left table, matches or NULL from the right	`... FROM customers c LEFT JOIN orders o ON o.customer_id = c.id`
CASE WHEN	If-then logic inside a query	`CASE WHEN amount > 1000 THEN 'High' ELSE 'Normal' END`
COALESCE	First non-NULL value	`COALESCE(phone, email, 'none')`
Subquery	A query inside another query	`WHERE amount > (SELECT AVG(amount) FROM orders)`
CTE (WITH)	Names a step so long queries read top to bottom	`WITH monthly AS (SELECT ...) SELECT * FROM monthly;`
ROW_NUMBER / RANK	Numbers rows within a group	`ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount DESC)`
Running total	Window SUM over an ordered set	`SUM(amount) OVER (ORDER BY order_date)`
LAG / LEAD	Value from the previous or next row	`LAG(revenue) OVER (ORDER BY month)`
UNION / UNION ALL	Stacks results; UNION removes duplicates	`SELECT city FROM a UNION ALL SELECT city FROM b;`
INSERT / UPDATE / DELETE	Changes data (use a WHERE on UPDATE and DELETE)	`UPDATE orders SET status = 'paid' WHERE id = 42;`
CREATE TABLE	Defines a table with types and keys	`CREATE TABLE t (id INT PRIMARY KEY, name TEXT NOT NULL);`
Index	Speeds up lookups on a column	`CREATE INDEX idx_orders_customer ON orders(customer_id);`
Transaction	Groups changes so they all succeed or all fail	`BEGIN; ... COMMIT; (or ROLLBACK;)`
Order of execution	FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT	`That is why a SELECT alias cannot be used in WHERE`

DAX (Power BI)

Measures recalculate in the current filter context. Prefer measures over calculated columns for totals.

Item	What it does	Example
Measure	A calculation evaluated for each visual's filters	`Total Sales = SUM(Sales[Amount])`
CALCULATE	Changes the filter context of an expression	`North Sales = CALCULATE([Total Sales], Region[Name] = "North")`

FILTER	Returns a filtered table, used inside CALCULATE or iterators	`CALCULATE([Total Sales], FILTER(Sales, Sales[Qty] > 5))`
ALL / REMOVEFILTERS	Ignores filters, for shares of total	`Share = DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Region)))`
DIVIDE	Safe division that handles divide by zero	`Margin % = DIVIDE([Profit], [Total Sales])`
SUMX / AVERAGEX	Iterates row by row, then aggregates	`Revenue = SUMX(Sales, Sales[Qty] * Sales[Price])`
RELATED	Brings a value from a related (one-side) table	`Sales[Category] = RELATED(Product[Category])`
TOTALYTD	Year-to-date total (needs a date table)	`Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])`
SAMEPERIODLASTYEAR	The same dates one year earlier	`Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))`
DATEADD	Shifts dates by days, months, quarters or years	`CALCULATE([Total Sales], DATEADD('Date'[Date], -1, MONTH))`
VAR / RETURN	Variables make measures readable and faster	`VAR ly = [Sales LY] RETURN DIVIDE([Total Sales] - ly, ly)`
Filter context	The filters a visual, slicer or row applies to a measure	`The same measure shows different numbers in each matrix cell`
Row context	The current row in a calculated column or iterator	`Exists inside SUMX and calculated columns`
Star schema	Fact table in the centre, dimension tables around it	`Sales (fact) linked to Date, Product and Customer (dimensions)`

Python essentials

Core Python syntax used in data work.

Item	What it does	Example
Variables and types	int, float, str, bool, None	`price = 499.0; city = "Pune"; paid = True`
f-strings	Put values inside text	`f"Total: {total:,.2f}"`
List	Ordered, changeable collection	`nums = [3, 1, 2]; nums.append(5)`
Dictionary	Key-value pairs	`row = {"id": 1, "city": "Pune"}; row["city"]`
Tuple / set	Fixed sequence; unique items	`point = (2, 3); ids = {1, 2, 2} # {1, 2}`
if / elif / else	Branching	`if score >= 90: grade = "A"`
for / while	Loops	`for i, x in enumerate(items): ...`
List comprehension	Build a list in one line	`[x * 2 for x in nums if x > 1]`
Functions	Reusable code with parameters and a return value	`def tax(amount, rate=0.18): return amount * rate`
Lambda	Small unnamed function	`sorted(rows, key=lambda r: r["amount"])`

try / except	Handle errors	`try: x = int(s) except ValueError: x = 0`
with open	Read or write files safely	`with open("data.csv") as f: text = f.read()`
Imports	Use libraries	`import pandas as pd; from pathlib import Path`
Classes	Bundle data and behaviour	`class Order: def __init__(self, amount): self.amount = amount`
Virtual environment	Isolated packages per project	`python -m venv .venv; pip install pandas`
Type hints	Document expected types	`def total(xs: list[float]) -> float: ...`

pandas

The main Python library for tables. df is a DataFrame.

Item	What it does	Example
Read data	Load CSV, Excel or Parquet	`df = pd.read_csv("sales.csv")`
Inspect	First rows, column types, summary	`df.head(); df.info(); df.describe()`
Select columns	One column or several	`df["city"]; df[["city", "amount"]]`
Filter rows	Boolean conditions	`df[(df.amount > 1000) & (df.city == "Pune")]`
loc / iloc	Select by label / by position	`df.loc[df.id == 5, "status"]; df.iloc[0:5, 0:3]`
New column	Calculate from other columns	`df["gst"] = df.amount * 0.18`
Missing values	Find, fill or drop	`df.isna().sum(); df.fillna(0); df.dropna()`
Duplicates	Find and remove	`df.drop_duplicates(subset=["order_id"])`
Types	Convert columns	`df["date"] = pd.to_datetime(df["date"])`
groupby	Aggregate per group	`df.groupby("city")["amount"].agg(["sum", "mean"])`
pivot_table	Spreadsheet-style pivot	`df.pivot_table(index="city", columns="month", values="amount", aggfunc="sum")`
merge	SQL-style joins	`orders.merge(customers, on="customer_id", how="left")`
concat	Stack DataFrames	`pd.concat([jan, feb], ignore_index=True)`
sort_values	Sort rows	`df.sort_values("amount", ascending=False)`
apply / map	Apply a function to values	`df["band"] = df.amount.map(lambda a: "High" if a > 1000 else "Low")`

Dates	Parts of dates and resampling	`df.date.dt.month; df.set_index("date").resample("M").sum()`
value_counts	Frequency of each value	`df.city.value_counts(normalize=True)`
Write data	Save results	`df.to_csv("out.csv", index=False); df.to_parquet("out.parquet")`

Statistics formulas

Formulas behind common analyses. n is the sample size.

Item	What it does	Example
Mean	Sum of values ÷ n	`(10 + 20 + 30) ÷ 3 = 20`
Median	Middle value when sorted; robust to outliers	`Median of 3, 5, 100 is 5`
Variance (sample)	Σ(x − mean)² ÷ (n − 1)	`Spread of the values around the mean`
Standard deviation	√variance, in the data's own units	`Delivery time 30 ± 6 minutes`
Percentile / IQR	Value below which a % of data falls; IQR = Q3 − Q1	`Outlier rule: below Q1 − 1.5×IQR or above Q3 + 1.5×IQR`
z-score	(x − mean) ÷ standard deviation	`z = 2 means two standard deviations above the mean`
Standard error of a mean	s ÷ √n	`s = 12, n = 144 gives 1.0`
Standard error of a proportion	√(p(1 − p) ÷ n)	`p = 0.8, n = 100 gives 0.04 (4 points)`
95% confidence interval	Estimate ± 1.96 × standard error (large samples)	`80% ± 7.8 points on 100 cases`
Correlation (r)	Strength of a straight-line relationship, −1 to 1	`r = 0.8 is strong; it does not prove cause`
p-value	Chance of a result at least this extreme if there were no real effect	`p < 0.05 is a common, not magic, threshold`
t-test	Compares means of two groups	`Average order value, app vs web`
Chi-square test	Tests whether two categorical variables are related	`Plan type vs churned (yes/no)`
Conversion rate	Conversions ÷ visitors	`120 ÷ 4,000 = 3%`
Relative lift	(new − old) ÷ old	`(3.3% − 3.0%) ÷ 3.0% = 10% lift`

ML metrics

How to score models. TP, FP, FN and TN are true positives, false positives, false negatives and true negatives.

Item	What it does	Example
Accuracy	(TP + TN) ÷ all	`Misleading when one class is rare`

Precision	TP ÷ (TP + FP): of predicted positives, how many were right	`32 ÷ 50 = 64%`
Recall (sensitivity)	TP ÷ (TP + FN): of actual positives, how many were found	`32 ÷ 40 = 80%`
F1 score	2 × precision × recall ÷ (precision + recall)	`2 × 0.64 × 0.80 ÷ 1.44 = 0.71`
Specificity	TN ÷ (TN + FP)	`Share of genuine non-fraud correctly cleared`
Confusion matrix	Table of TP, FP, FN, TN	`Read it before any single metric`
ROC-AUC	How well scores rank positives above negatives, 0.5 (random) to 1	`0.85 means good ranking across thresholds`
PR-AUC	Area under the precision-recall curve; better for rare positives	`Fraud, rare defects`
Log loss	Penalises confident wrong probabilities	`Lower is better`
MAE	Mean absolute error: average size of errors	`Off by ₹120 on average`
RMSE	Root mean squared error: punishes large errors more	`RMSE ≥ MAE always`
MAPE	Mean absolute percentage error	`Avoid when actual values can be near zero`
R-squared	Share of variance explained (can be negative on new data)	`0.72 explains 72% of the variation`
Cross-validation	Train and test on several splits for a stable estimate	`5-fold: five train/test rounds`
Overfitting check	Compare training and validation scores	`Train 0.99, validation 0.70 means overfitting`

APIs, webhooks and MCP

Terms used when connecting tools and AI assistants to data.

Item	What it does	Example
API	A defined way for programs to ask another program for data or actions	`The weather app calls a weather API`
Endpoint	A URL that does one job	`GET /v1/orders/{id}`
HTTP methods	GET reads, POST creates, PUT/PATCH updates, DELETE removes	`POST /v1/orders creates an order`
Status codes	200 OK, 201 Created, 400 bad request, 401 not logged in, 403 not allowed, 404 not found, 429 too many requests, 500 server error	`Retry 429 and 5xx with backoff; fix 4xx`
Headers	Metadata sent with a request	`Authorization: Bearer <token>; Content-Type: application/json`
JSON	The usual text format for API data	`{"id": 42, "status": "paid"}`
Query parameters	Options in the URL	`/orders?status=paid&page=2`
Authentication	Proving who is calling: API key, OAuth token	`Never put keys in code or prompts`

OAuth 2.0	Lets a user grant an app limited access without sharing a password	`Sign in with Google, then allow read-only calendar`
Rate limit	Maximum calls in a time window	`100 requests per minute`
Pagination	Results returned in pages	`Follow next_cursor until it is empty`
Webhook	The other app calls your URL when something happens	`Payment succeeded sends a POST to your endpoint`
Idempotency	Repeating a request has no extra effect	`Send an Idempotency-Key so a retried payment is not charged twice`
OpenAPI	A standard document describing an API	`Swagger UI reads it and shows try-it buttons`
MCP	Model Context Protocol: an open standard for connecting AI assistants to tools and data	`An MCP server lets an assistant read a database or a drive`
Access token	A temporary credential sent with each request instead of a password	`Authorization: Bearer eyJ... (expires, then refreshed)`
MCP host / client / server	Host: the AI app the user works in; client: its connection to one server; server: the program exposing tools and data	`Host: Claude Desktop; server: a Postgres MCP server`
MCP tool / resource / prompt	Actions the model can call; data it can read; reusable prompt templates	`Tool: create_ticket; resource: a policy file`

AI and LLM terms

Words you will meet when building or evaluating GenAI features.

Item	What it does	Example
LLM	Large language model: predicts the next token from huge amounts of text	`Claude, GPT, Gemini, Llama`
Token	A piece of text the model reads and writes (roughly ¾ of an English word)	`Prices are per million tokens`
Context window	The maximum tokens a model can consider at once	`Instructions + documents + chat + answer must fit`
Prompt / system prompt	The request; the standing instructions for every request	`Role, rules, format, examples`
Temperature	Randomness of the output; lower is more predictable	`Use low values for extraction`
Embedding	Numbers that capture meaning, used for similarity search	`"refund" and "money back" land close together`
Vector database	Stores embeddings and finds the nearest ones	`pgvector, Pinecone, Chroma`
RAG	Retrieval-augmented generation: search your documents, then answer from them	`Help-centre assistant with citations`
Chunking	Splitting documents into passages for retrieval	`One help-article section per chunk`
Hallucination	Confident output not supported by facts or sources	`An invented policy clause`

Tool use / function calling	The model asks your code to run a function	`get_order_status(order_id)`
Agent	An LLM that plans and uses tools in a loop toward a goal	`Researches, drafts and files a report`
Fine-tuning	Further training a model on your examples	`Usually after prompting and RAG are tried`
Guardrails	Checks on inputs and outputs	`Block personal data, validate JSON`
Eval	A repeatable test of AI quality on fixed cases	`Pass rate on a 200-case golden set`
Prompt injection	Text in the context that tries to override the instructions	`"Ignore your rules" hidden in an email`

Git and the command line

Everyday commands. Check your organisation's branching rules.

Item	What it does	Example
git clone	Copy a repository to your computer	`git clone https://github.com/org/repo.git`
git status / diff	See what changed	`git status; git diff`
git add / commit	Stage and save a change with a message	`git add file.py; git commit -m "Fix tax rounding"`
git branch / switch	Work on a separate line of changes	`git switch -c feature/report`
git pull / push	Get others' changes; share yours	`git pull; git push -u origin feature/report`
git log	History of commits	`git log --oneline -10`
Pull request	Ask for review before merging	`Opened on GitHub or GitLab`
pwd / ls / cd	Where am I; list files; change folder	`cd data; ls -l`
mkdir / cp / mv / rm	Make folder; copy; move or rename; delete (careful)	`mv old.csv archive/`
head / tail / wc	Look at the start or end of a file; count lines	`head -5 sales.csv; wc -l sales.csv`
grep	Search text in files	`grep -i "error" app.log`
Pipes and redirects	Send output to another command or a file	`grep ERROR app.log | wc -l > errors.txt`